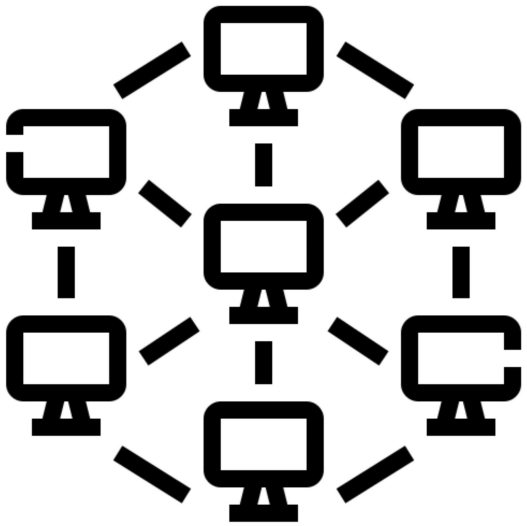
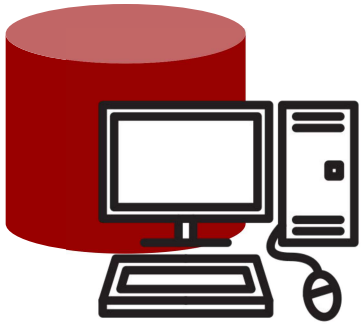
Relational Verification of Configurable Programs

Eric Hayden Campbell

joint work with Mark Barbone and Nate Foster



Computer Networks

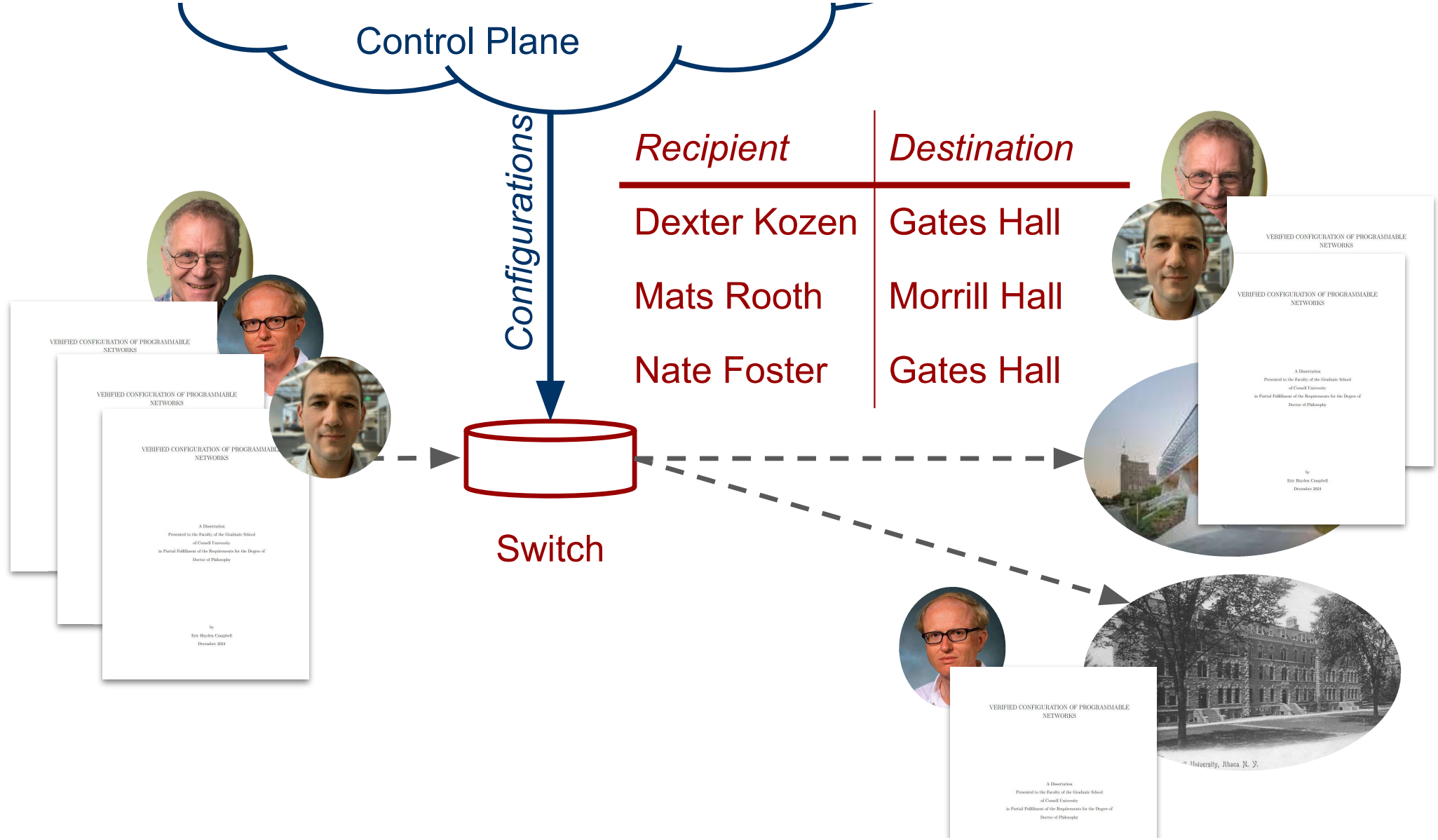


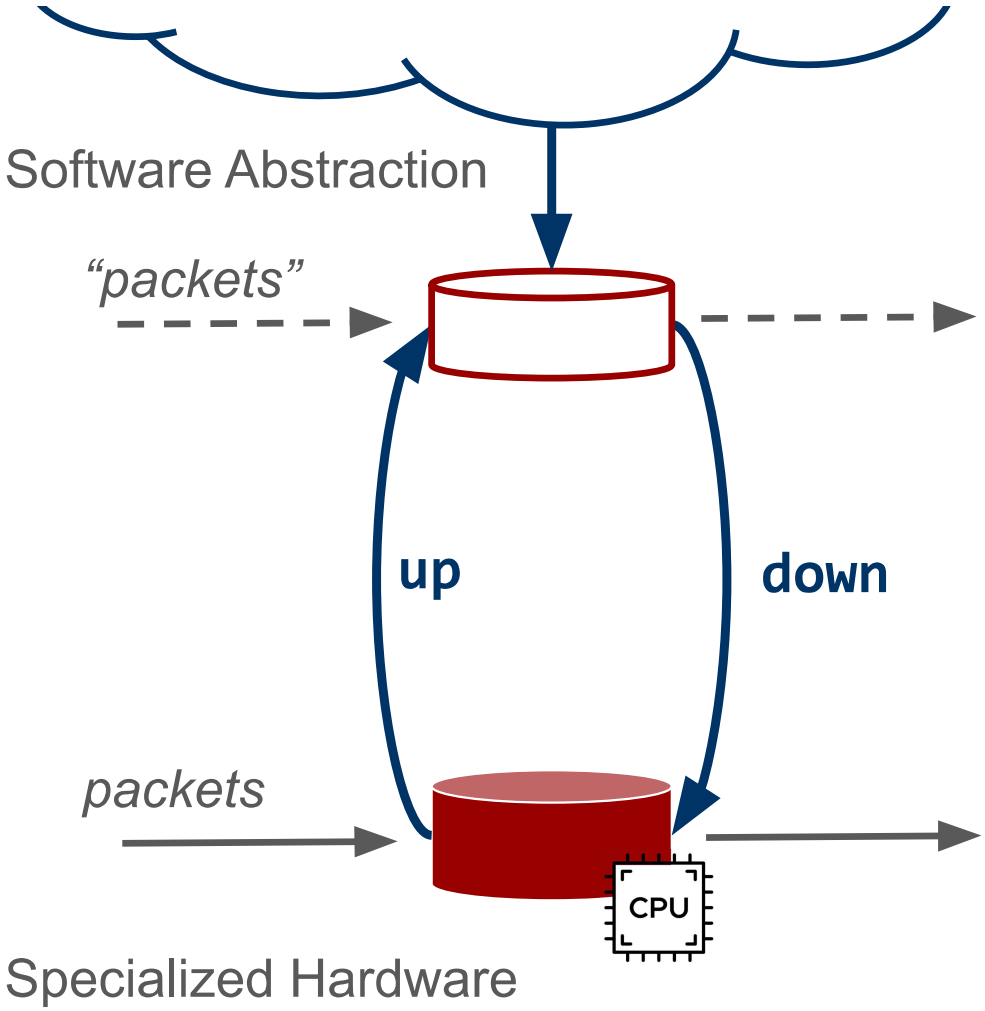
Database Applications



Security

code(*cfg'*) : $(A \rightarrow B)$


Control Plane
Configurations
Recipient
Destination
Dexter Kozen
Gates Hall
Mats Rooth
Morrill Hall
Nate Foster
Gates Hall
Switch
VERIFIED CONFIGURATION OF PROGRAMMABLE NETWORKS
A Dissertation
Presented to the Faculty of the Graduate School
of Cornell University
in Partial Fulfillment of the Requirements for the Degree of
Doctor of Philosophy
by
Eric Hayden Campbell
December 2024

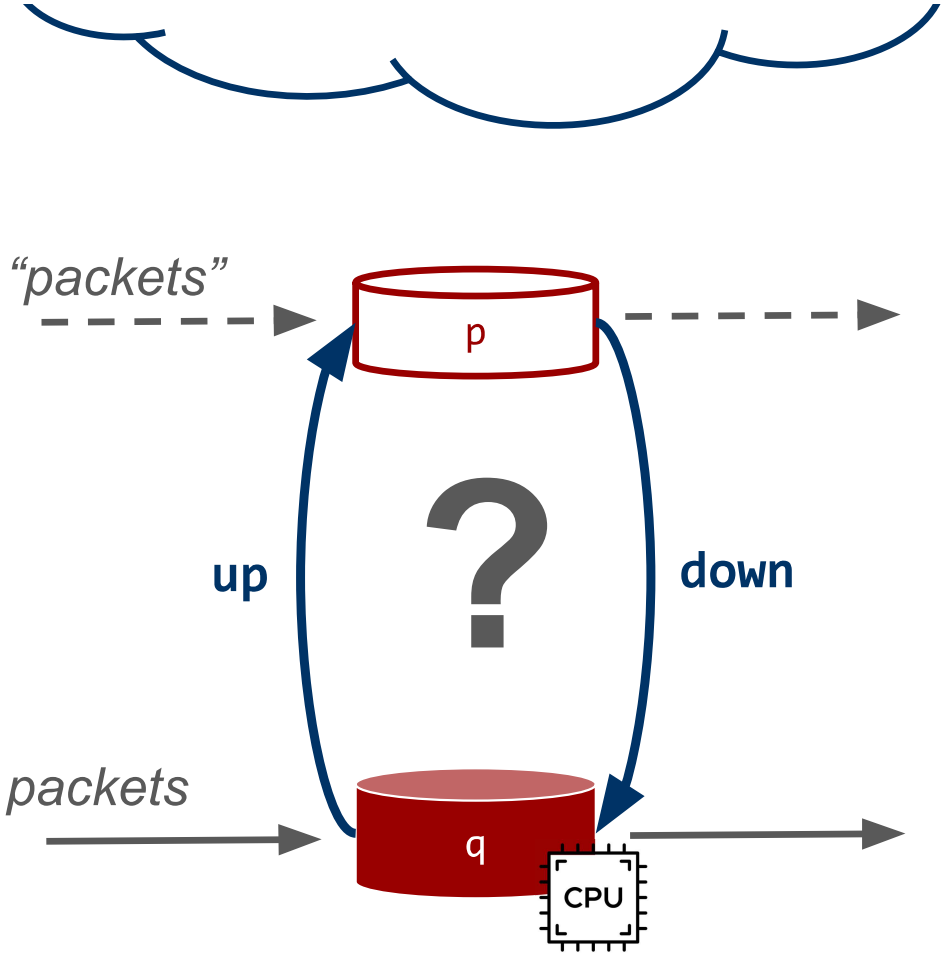


Recipient	*Destination*
Dexter Kozen	Gates Hall
Mats Rooth	Morill Hall
Nate Foster	Gates Hall

up

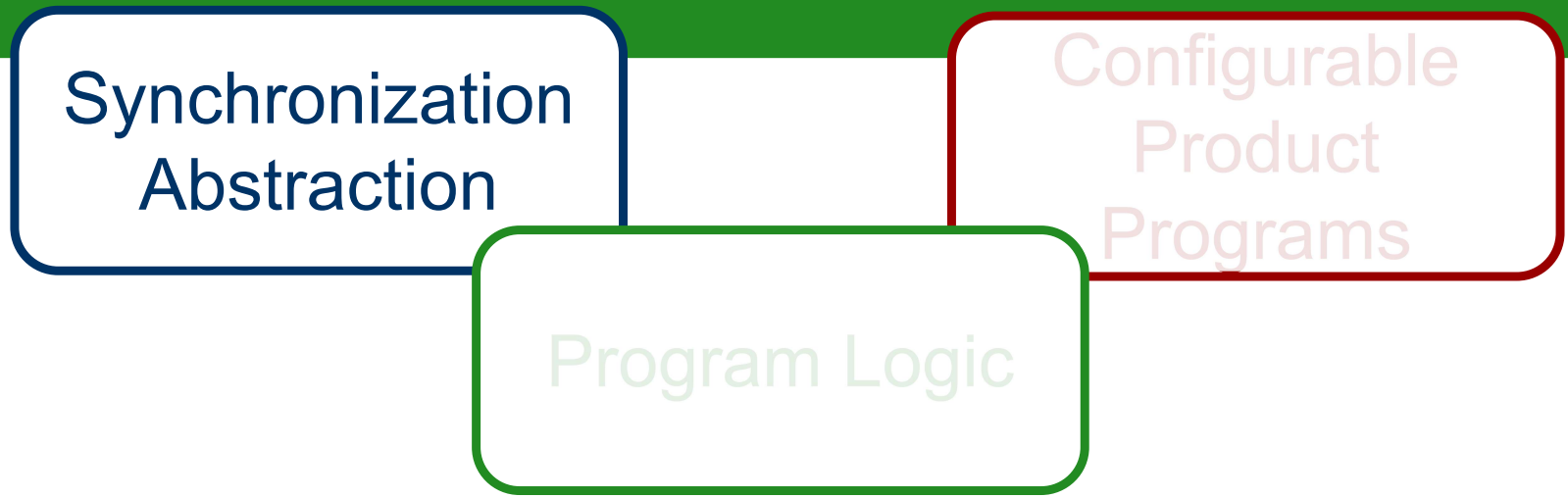
down

Recipient	*Dept*	*Dept*	*Destination*
Dexter Kozen	CS	CS	Gates Hall
Mats Rooth	Linguistics	Linguistics	Morrill Hall
Nate Foster	CS		

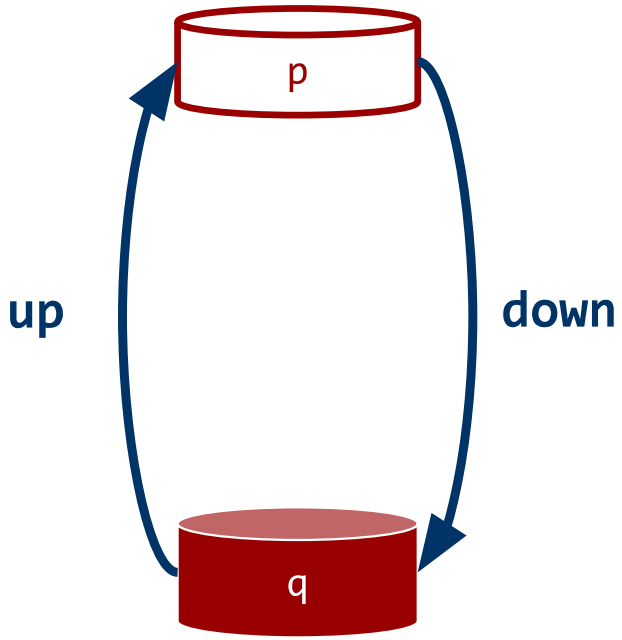


Correctness?

Relational Verification of Synchronized Configurable Programs



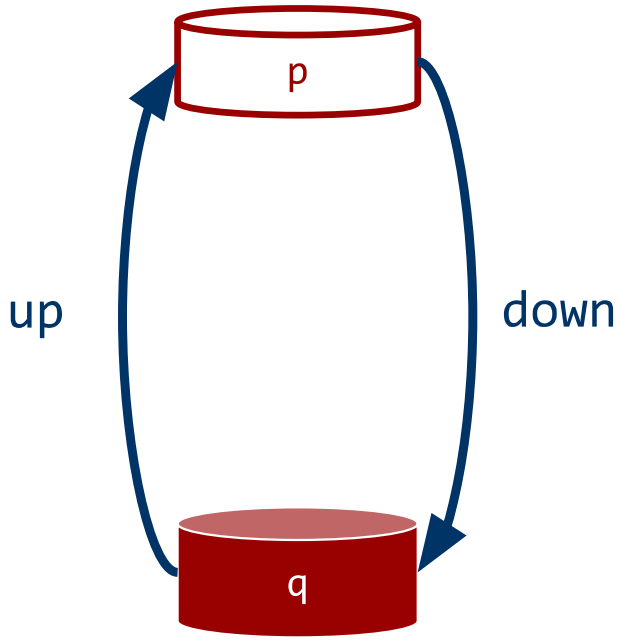
There and back again




```
(down; up) == id

(up; down) == id
```

Symmetric lenses (briefly)



State State

⟨cfg’, **k’**⟩= down **cfg** k ⇒
⟨ ⟩ cfg’ **k’**

⟨cfg, **k**⟩= up **cfg’** k’ ⇒
⟨**cfg’**, **k**⟩= down cfg **k**

Lens-Configured Equality

$\vartheta \subseteq$ Cfg $\times$ Cfg'

Lens Invariant

$\mathtt{l} \models \vartheta$

$\forall$(cfg,cfg') $\in \vartheta$. $\forall$ pkt.



L

Recipient	*Destination*
Dexter Kozen	Gates Hall
Mats Rooth	Morrill Hall
Nate Foster	Gates Hall





$\models L = D \circ R$

R

Recipient	*Dept*
Dexter Kozen	CS
Mats Rooth	Linguistics
Nate Foster	CS

D

Dept	*Destination*
CS	Gates Hall
Linguistics	Morrill Hall

Lens Maintains an Invariant

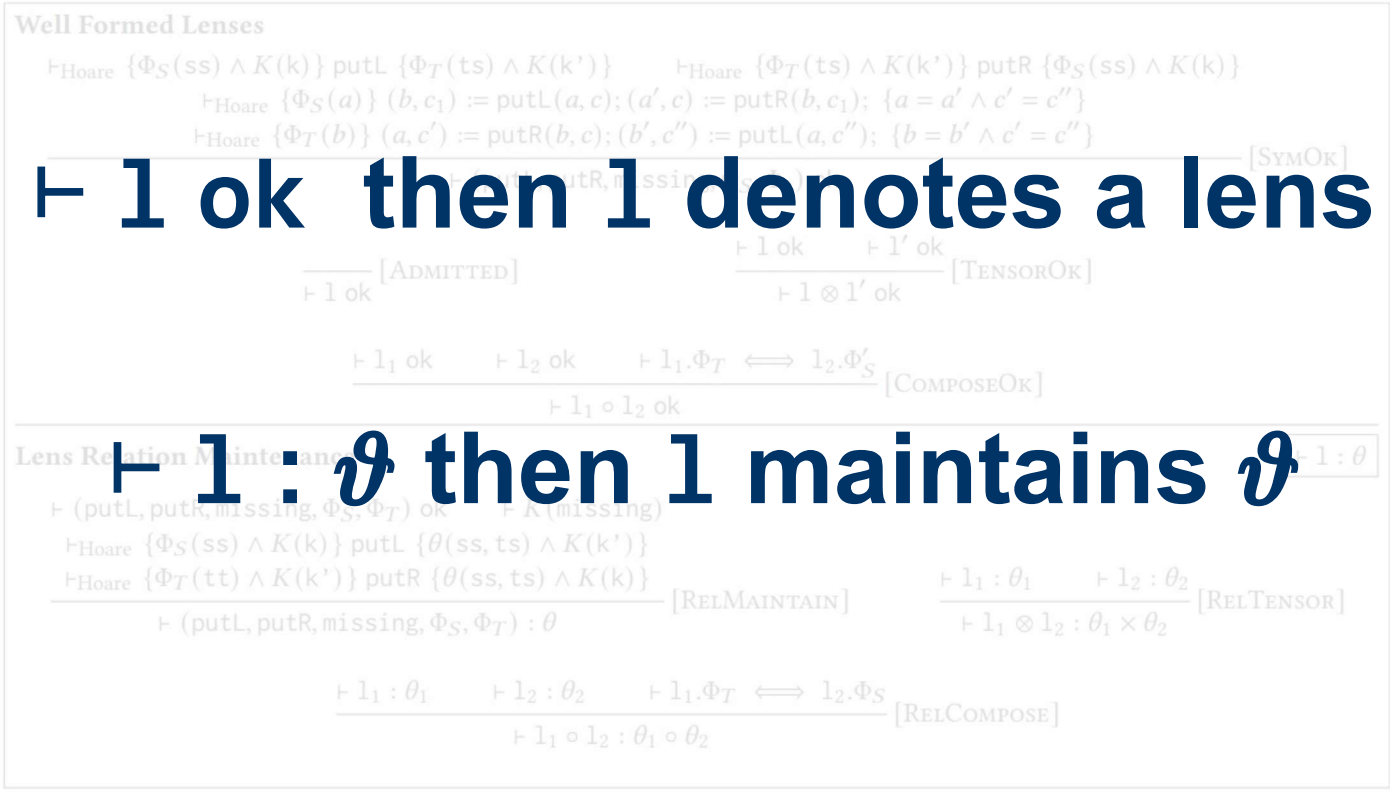
$$\forall \texttt{cfg, cfg', k, k'.}$$

$$\texttt{cfg',k' = down(cfg, k)} \Rightarrow \texttt{cfg} \ \vartheta \ \texttt{cfg'}$$

$$\forall \texttt{cfg, cfg', k, k'.}$$

$$\texttt{cfg,k = up(cfg', k')} \Rightarrow \texttt{cfg} \ \vartheta \ \texttt{cfg'}$$

$$\langle \text{down, up} \rangle \vDash \vartheta$$

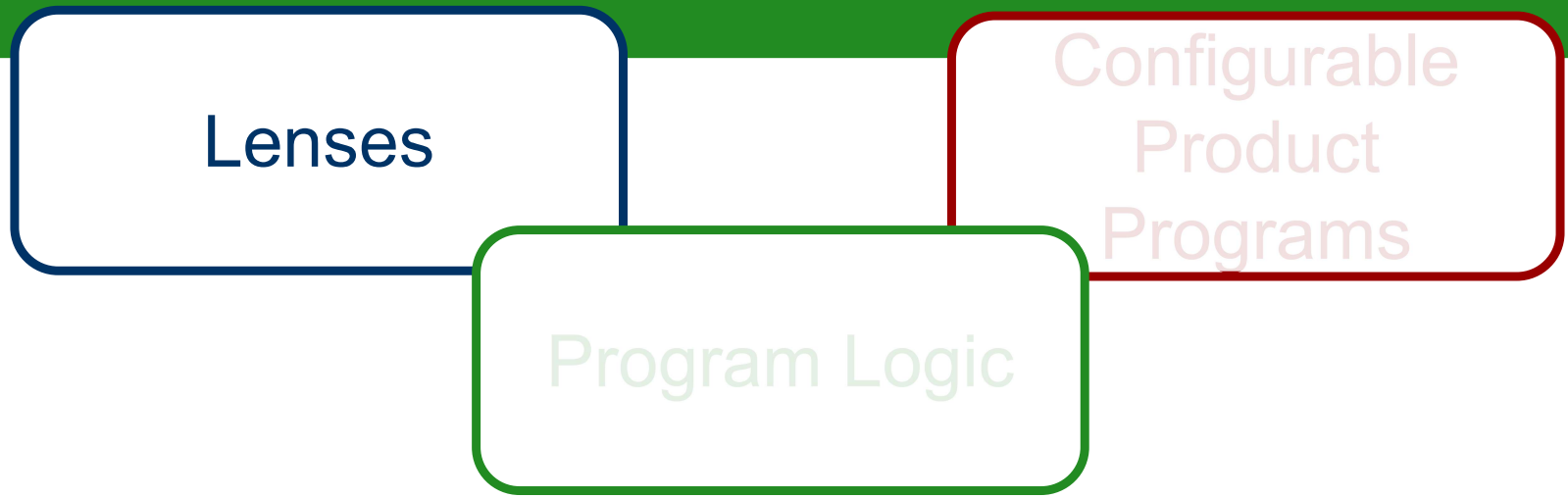


⊢ l ok then l denotes a lens

⊢ l : ϑ then l maintains ϑ

Fig. 3. Proof Rules for Lenses and their Relations

Relational Program Logic verifying **synchronization** of **configurable programs**



Product Programs + Lenses

```
p ::= skip
    | p; p
    | above c | below c
    | if b p p
    | while b p
    | <F, G> ← lens l in p
```

Lists of Configuration Variables

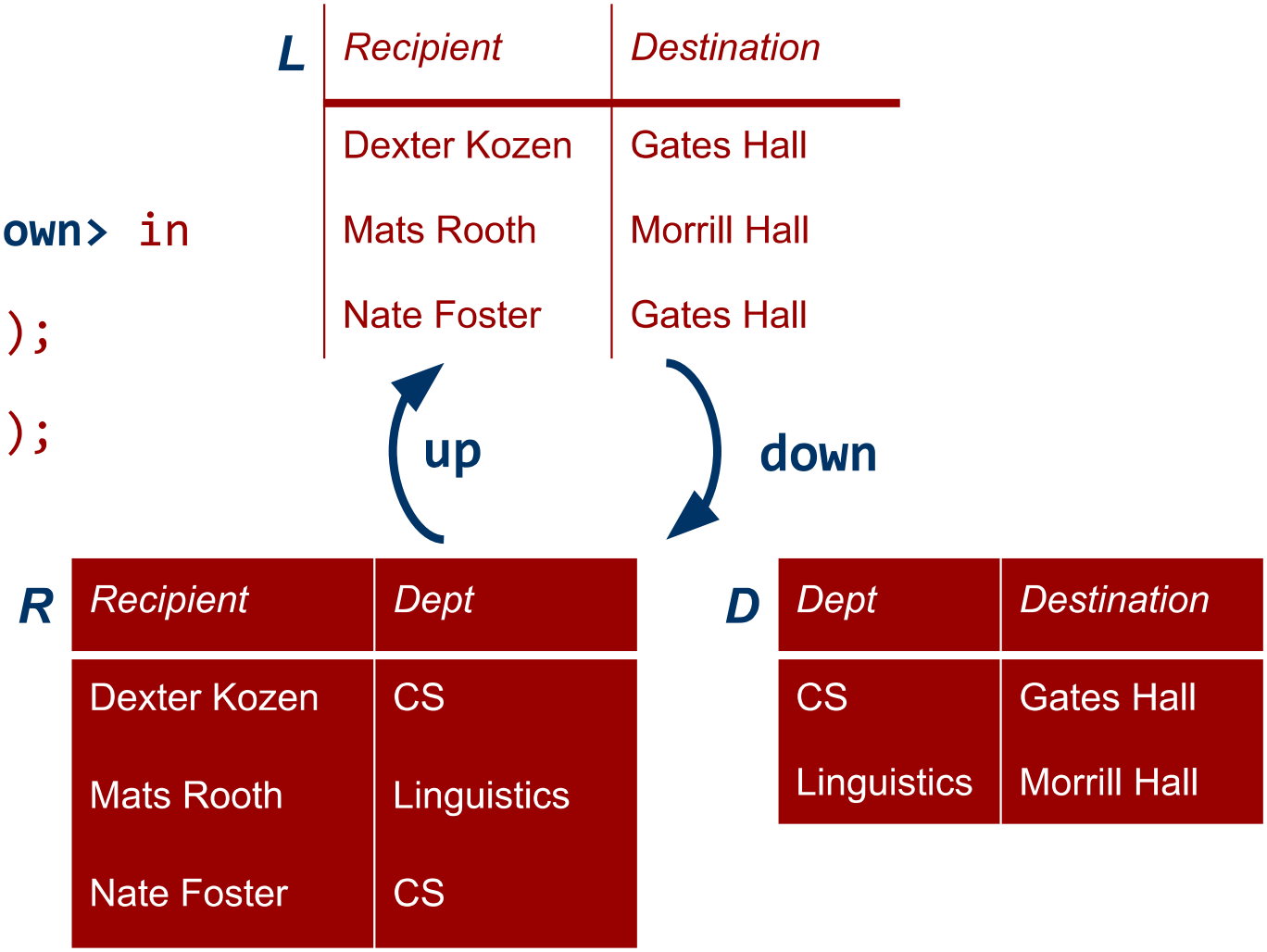
lens

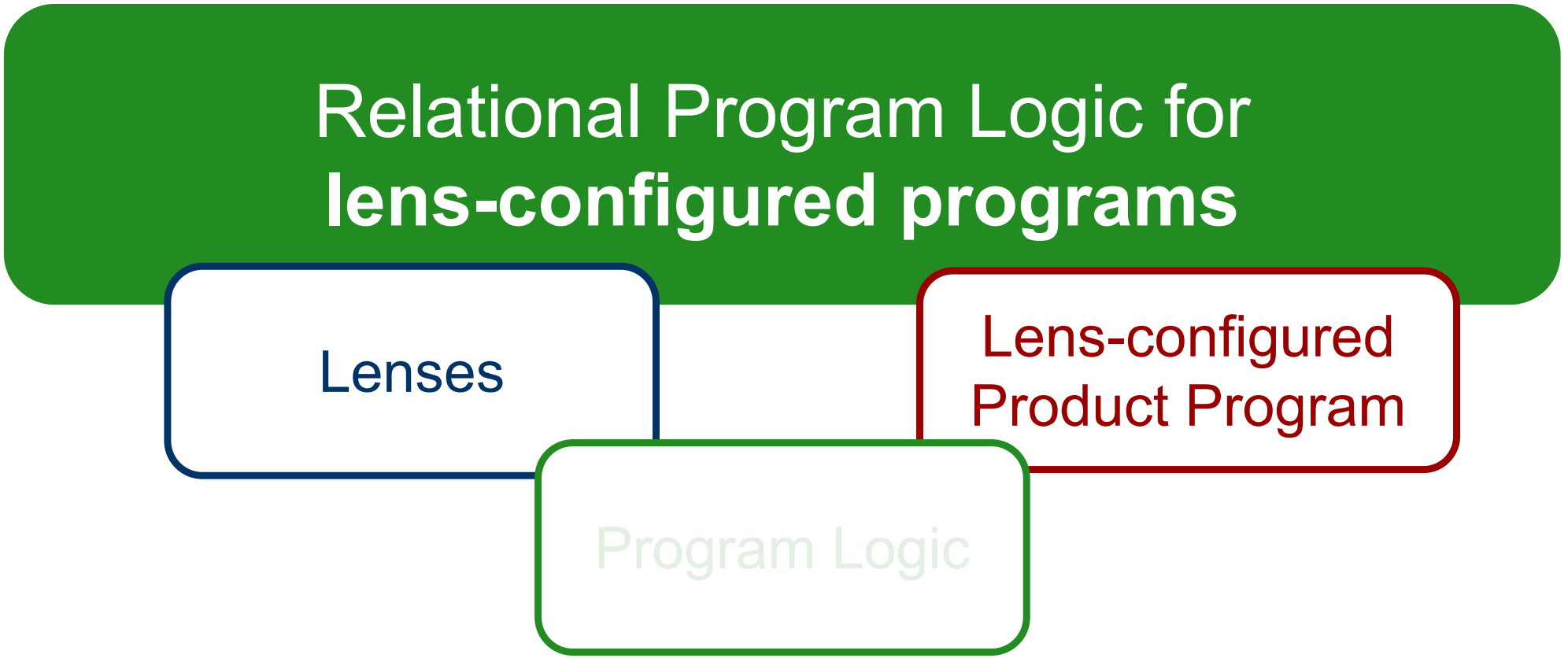
Example

```
<L; D,R> <- lens <up,down> in

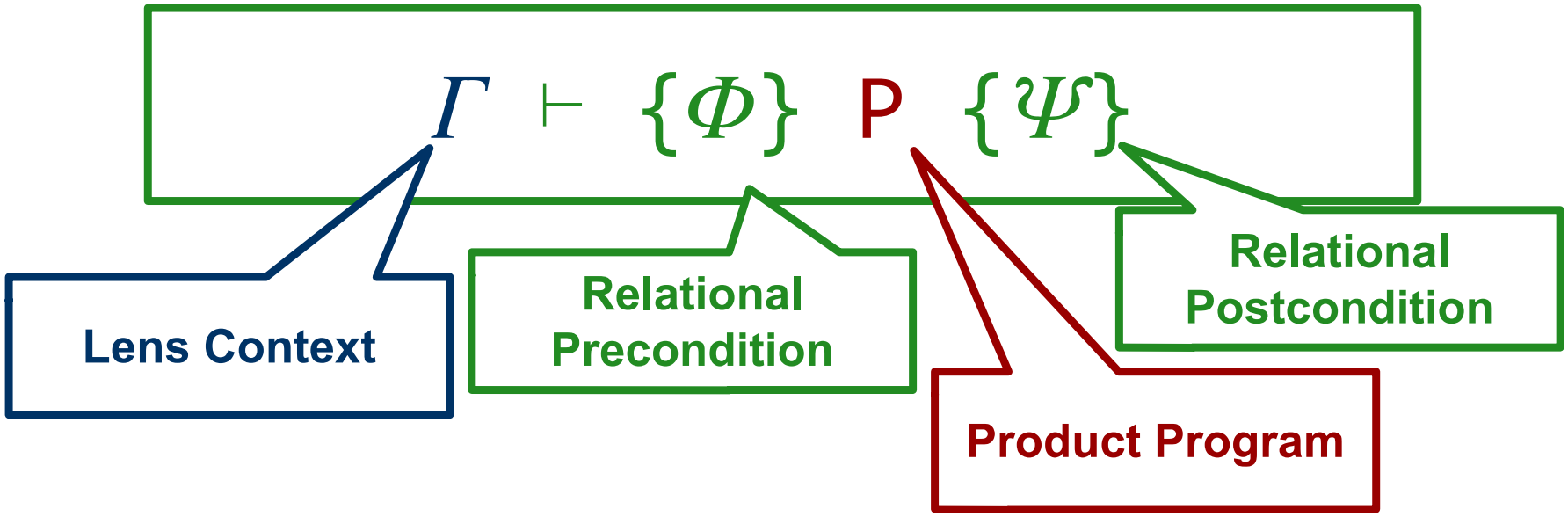
above dest := L(member);

below dept := R(member);
      dest := D(dept)
```




Relational Program Logic for
lens-configured programs
Lenses
Lens-configured
Product Program
Program Logic

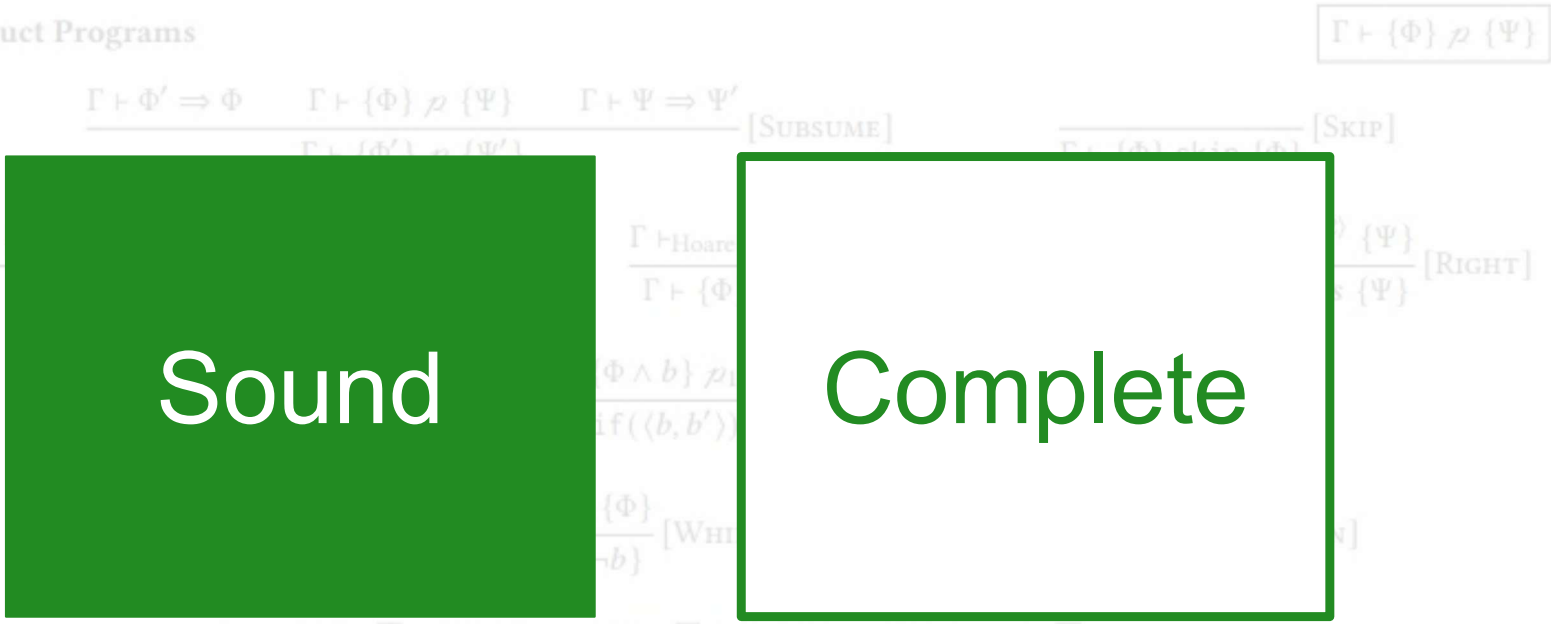
Relational Program Logics + Lenses



Binding lenses one at a time

$$\frac{\vdash \texttt{l}\ \text{ok} \qquad \vdash \texttt{l} : \vartheta(\texttt{F}, \texttt{G}) \qquad \Gamma, (\texttt{F}, \texttt{G}) : \vartheta \vdash \{\Phi\}\ \texttt{p}\ \{\Psi\}}{\Gamma \vdash \{\Phi\}\ \texttt{<F, G>} \leftarrow \texttt{lens l in p}\ \{\Psi\}}$$

Program Logic for Lens-configured Product Programs



$member_1 = member_2$

```
<L; D,R> <- lens <up,down> in
```

$L(member_1) = R(D(member_2))$

```
above dest := L(member);
```

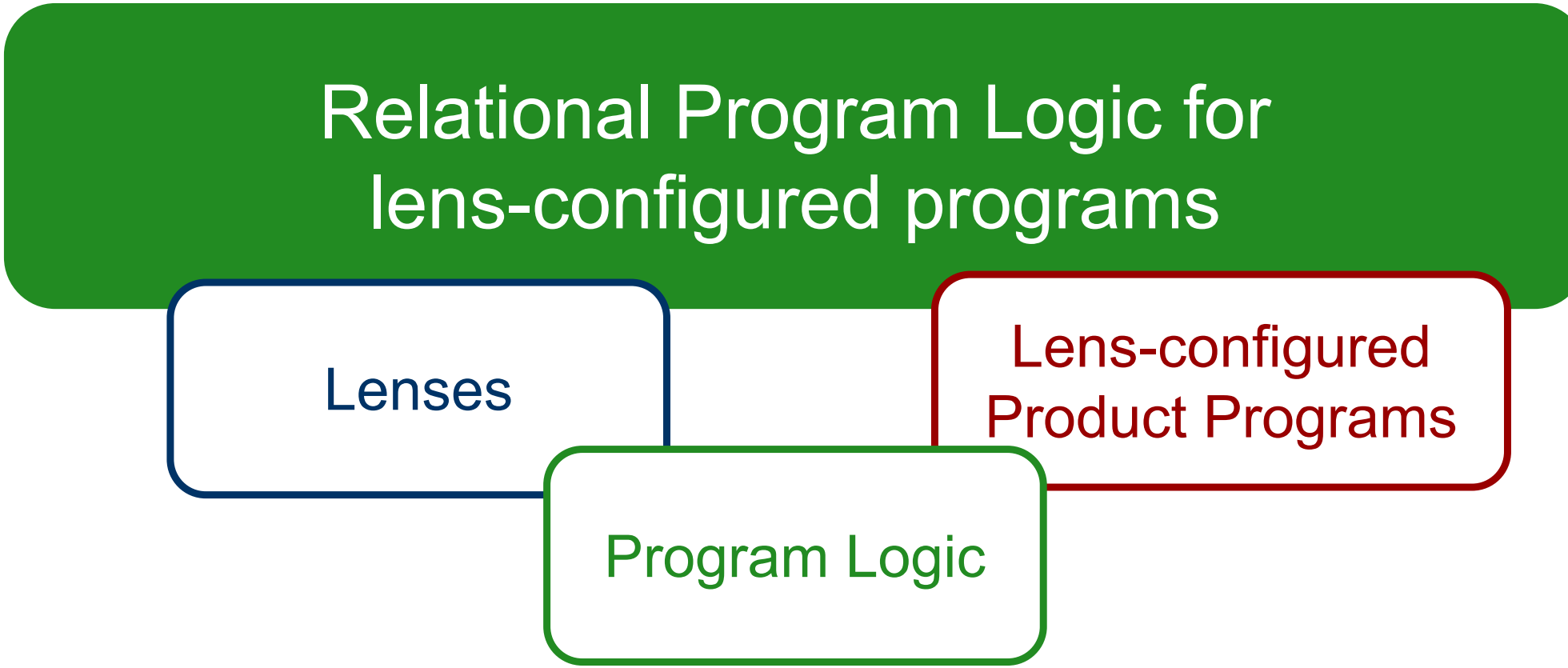
$dest_1 = R(D(member_2))$

```
below dept := R(member);
      dest := D(dept)
```

$dest_1 = dest_2$

$\forall m.\ L(m) = R(D(m))$



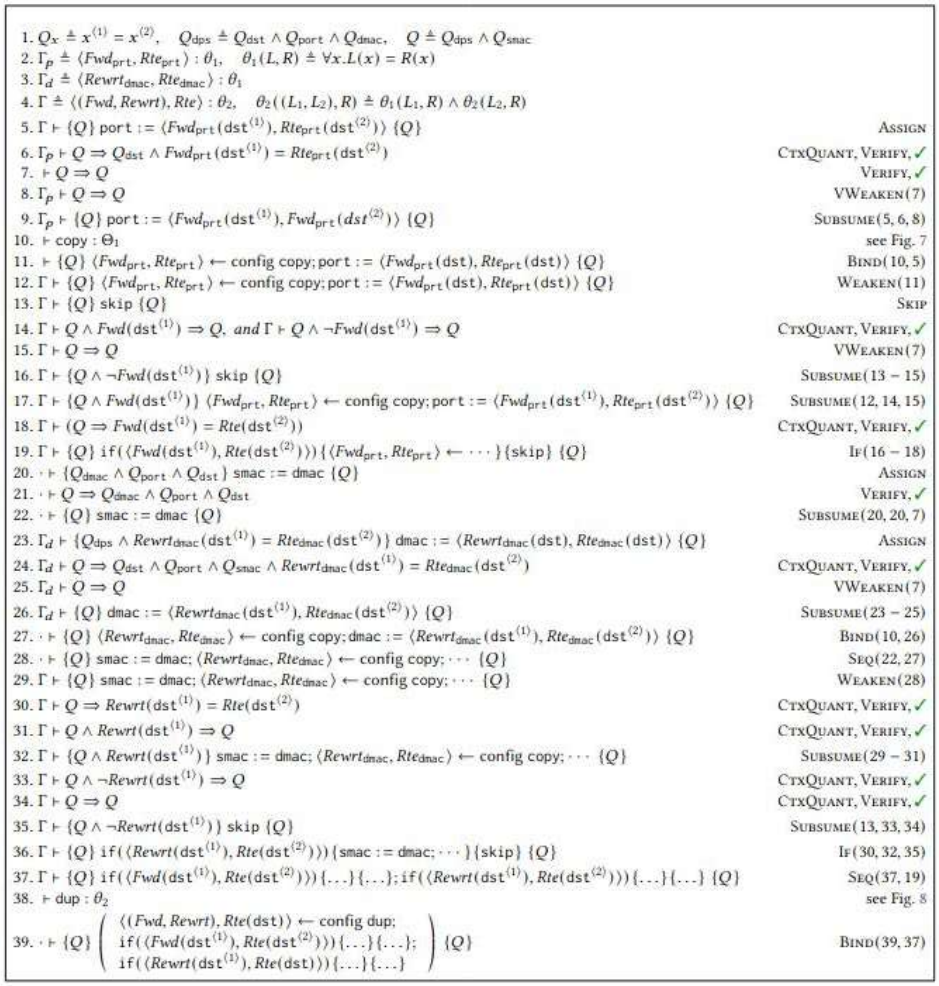

Relational Program Logic for lens-configured programs
Lenses
Lens-configured Product Programs
Program Logic


Bifocal

Spectacle

```
assume_ eq@@lens (copy Nxt Rte) @@
annotate eq@@ifthen2 (Nxt dst, Rte dst)(
 lens (compose Nxt_port Grp Rte_prt) @@
 assume_ eq @@
 assign2 port (Nxt_port dst, Rte_prt dst) @@
 assign1 smac dmac @@
 lens (copy Nxt_dmac Rte_dmac) @@
  assume_ eq @@
  assign2 dmac (Nxt_dmac dst, Rte_dmac dst)
  skip) @@ skip
assert_ eq
```

Combinator Library Proof



1. $Q_x \triangleq x^{\langle 1\rangle} = x^{\langle 2\rangle}, \quad Q_{\mathsf{dps}} \triangleq Q_{\mathsf{dst}} \wedge Q_{\mathsf{port}} \wedge Q_{\mathsf{dmac}}, \quad Q \triangleq Q_{\mathsf{dps}} \wedge Q_{\mathsf{smac}}$
2. $\Gamma_p \triangleq \langle Fwd_{\mathsf{prt}}, Rte_{\mathsf{prt}}\rangle : \theta_1, \quad \theta_1(L,R) \triangleq \forall x.L(x) = R(x)$
3. $\Gamma_d \triangleq \langle Rewrt_{\mathsf{dmac}}, Rte_{\mathsf{dmac}}\rangle : \theta_1$
4. $\Gamma \triangleq \langle (Fwd, Rewrt), Rte\rangle : \theta_2, \quad \theta_2((L_1,L_2),R) \triangleq \theta_1(L_1,R) \wedge \theta_2(L_2,R)$
5. $\Gamma \vdash \{Q\}\ \mathtt{port} := \langle Fwd_{\mathsf{prt}}(\mathtt{dst}^{\langle 1\rangle}), Rte_{\mathsf{prt}}(\mathtt{dst}^{\langle 2\rangle})\rangle\ \{Q\}$ — Assign
6. $\Gamma_p \vdash Q \Rightarrow Q_{\mathsf{dst}} \wedge Fwd_{\mathsf{prt}}(\mathtt{dst}^{\langle 1\rangle}) = Rte_{\mathsf{prt}}(\mathtt{dst}^{\langle 2\rangle})$ — CtxQuant, Verify, ✓
7. $\vdash Q \Rightarrow Q$ — Verify, ✓
8. $\Gamma_p \vdash Q \Rightarrow Q$ — VWeaken(7)
9. $\Gamma_p \vdash \{Q\}\ \mathtt{port} := \langle Fwd_{\mathsf{prt}}(\mathtt{dst}^{\langle 1\rangle}), Fwd_{\mathsf{prt}}(dst^{\langle 2\rangle})\rangle\ \{Q\}$ — Subsume(5, 6, 8)
10. $\vdash \mathtt{copy} : \Theta_1$ — see Fig. 7
11. $\vdash \{Q\}\ \langle Fwd_{\mathsf{prt}}, Rte_{\mathsf{prt}}\rangle \leftarrow \mathtt{config\ copy}; \mathtt{port} := \langle Fwd_{\mathsf{prt}}(\mathtt{dst}), Rte_{\mathsf{prt}}(\mathtt{dst})\rangle\ \{Q\}$ — Bind(10, 5)
12. $\Gamma \vdash \{Q\}\ \langle Fwd_{\mathsf{prt}}, Rte_{\mathsf{prt}}\rangle \leftarrow \mathtt{config\ copy}; \mathtt{port} := \langle Fwd_{\mathsf{prt}}(\mathtt{dst}), Rte_{\mathsf{prt}}(\mathtt{dst})\rangle\ \{Q\}$ — Weaken(11)
13. $\Gamma \vdash \{Q\}\ \mathtt{skip}\ \{Q\}$ — Skip
14. $\Gamma \vdash Q \wedge Fwd(\mathtt{dst}^{\langle 1\rangle}) \Rightarrow Q$, *and* $\Gamma \vdash Q \wedge \neg Fwd(\mathtt{dst}^{\langle 1\rangle}) \Rightarrow Q$ — CtxQuant, Verify, ✓
15. $\Gamma \vdash Q \Rightarrow Q$ — VWeaken(7)
16. $\Gamma \vdash \{Q \wedge \neg Fwd(\mathtt{dst}^{\langle 1\rangle})\}\ \mathtt{skip}\ \{Q\}$ — Subsume(13 – 15)
17. $\Gamma \vdash \{Q \wedge Fwd(\mathtt{dst}^{\langle 1\rangle})\}\ \langle Fwd_{\mathsf{prt}}, Rte_{\mathsf{prt}}\rangle \leftarrow \mathtt{config\ copy}; \mathtt{port} := \langle Fwd_{\mathsf{prt}}(\mathtt{dst}^{\langle 1\rangle}), Rte_{\mathsf{prt}}(\mathtt{dst}^{\langle 2\rangle})\rangle\ \{Q\}$ — Subsume(12, 14, 15)
18. $\Gamma \vdash (Q \Rightarrow Fwd(\mathtt{dst}^{\langle 1\rangle}) = Rte(\mathtt{dst}^{\langle 2\rangle}))$ — CtxQuant, Verify, ✓
19. $\Gamma \vdash \{Q\}\ \mathtt{if}(\langle Fwd(\mathtt{dst}^{\langle 1\rangle}), Rte(\mathtt{dst}^{\langle 2\rangle})\rangle)\{\langle Fwd_{\mathsf{prt}}, Rte_{\mathsf{prt}}\rangle \leftarrow \cdots\}\{\mathtt{skip}\}\ \{Q\}$ — If(16 – 18)
20. $\cdot \vdash \{Q_{\mathsf{dmac}} \wedge Q_{\mathsf{port}} \wedge Q_{\mathsf{dst}}\}\ \mathtt{smac} := \mathtt{dmac}\ \{Q\}$ — Assign
21. $\cdot \vdash Q \Rightarrow Q_{\mathsf{dmac}} \wedge Q_{\mathsf{port}} \wedge Q_{\mathsf{dst}}$ — Verify, ✓
22. $\cdot \vdash \{Q\}\ \mathtt{smac} := \mathtt{dmac}\ \{Q\}$ — Subsume(20, 20, 7)
23. $\Gamma_d \vdash \{Q_{\mathsf{dps}} \wedge Rewrt_{\mathsf{dmac}}(\mathtt{dst}^{\langle 1\rangle}) = Rte_{\mathsf{dmac}}(\mathtt{dst}^{\langle 2\rangle})\}\ \mathtt{dmac} := \langle Rewrt_{\mathsf{dmac}}(\mathtt{dst}), Rte_{\mathsf{dmac}}(\mathtt{dst})\rangle\ \{Q\}$ — Assign
24. $\Gamma_d \vdash Q \Rightarrow Q_{\mathsf{dst}} \wedge Q_{\mathsf{port}} \wedge Q_{\mathsf{smac}} \wedge Rewrt_{\mathsf{dmac}}(\mathtt{dst}^{\langle 1\rangle}) = Rte_{\mathsf{dmac}}(\mathtt{dst}^{\langle 2\rangle})$ — CtxQuant, Verify, ✓
25. $\Gamma_d \vdash Q \Rightarrow Q$ — VWeaken(7)
26. $\Gamma_d \vdash \{Q\}\ \mathtt{dmac} := \langle Rewrt_{\mathsf{dmac}}(\mathtt{dst}^{\langle 1\rangle}), Rte_{\mathsf{dmac}}(\mathtt{dst}^{\langle 2\rangle})\rangle\ \{Q\}$ — Subsume(23 – 25)
27. $\cdot \vdash \{Q\}\ \langle Rewrt_{\mathsf{dmac}}, Rte_{\mathsf{dmac}}\rangle \leftarrow \mathtt{config\ copy}; \mathtt{dmac} := \langle Rewrt_{\mathsf{dmac}}(\mathtt{dst}^{\langle 1\rangle}), Rte_{\mathsf{dmac}}(\mathtt{dst}^{\langle 2\rangle})\rangle\ \{Q\}$ — Bind(10, 26)
28. $\cdot \vdash \{Q\}\ \mathtt{smac} := \mathtt{dmac}; \langle Rewrt_{\mathsf{dmac}}, Rte_{\mathsf{dmac}}\rangle \leftarrow \mathtt{config\ copy}; \cdots\ \{Q\}$ — Seq(22, 27)
29. $\Gamma \vdash \{Q\}\ \mathtt{smac} := \mathtt{dmac}; \langle Rewrt_{\mathsf{dmac}}, Rte_{\mathsf{dmac}}\rangle \leftarrow \mathtt{config\ copy}; \cdots\ \{Q\}$ — Weaken(28)
30. $\Gamma \vdash Q \Rightarrow Rewrt(\mathtt{dst}^{\langle 1\rangle}) = Rte(\mathtt{dst}^{\langle 2\rangle})$ — CtxQuant, Verify, ✓
31. $\Gamma \vdash Q \wedge Rewrt(\mathtt{dst}^{\langle 1\rangle}) \Rightarrow Q$ — CtxQuant, Verify, ✓
32. $\Gamma \vdash \{Q \wedge Rewrt(\mathtt{dst}^{\langle 1\rangle})\}\ \mathtt{smac} := \mathtt{dmac}; \langle Rewrt_{\mathsf{dmac}}, Rte_{\mathsf{dmac}}\rangle \leftarrow \mathtt{config\ copy}; \cdots\ \{Q\}$ — Subsume(29 – 31)
33. $\Gamma \vdash Q \wedge \neg Rewrt(\mathtt{dst}^{\langle 1\rangle}) \Rightarrow Q$ — CtxQuant, Verify, ✓
34. $\Gamma \vdash Q \Rightarrow Q$ — CtxQuant, Verify, ✓
35. $\Gamma \vdash \{Q \wedge \neg Rewrt(\mathtt{dst}^{\langle 1\rangle})\}\ \mathtt{skip}\ \{Q\}$ — Subsume(13, 33, 34)
36. $\Gamma \vdash \{Q\}\ \mathtt{if}(\langle Rewrt(\mathtt{dst}^{\langle 1\rangle}), Rte(\mathtt{dst}^{\langle 2\rangle})\rangle)\{\mathtt{smac} := \mathtt{dmac}; \cdots\}\{\mathtt{skip}\}\ \{Q\}$ — If(30, 32, 35)
37. $\Gamma \vdash \{Q\}\ \mathtt{if}(\langle Fwd(\mathtt{dst}^{\langle 1\rangle}), Rte(\mathtt{dst}^{\langle 2\rangle})\rangle)\{\ldots\}\{\ldots\}; \mathtt{if}(\langle Rewrt(\mathtt{dst}^{\langle 1\rangle}), Rte(\mathtt{dst}^{\langle 2\rangle})\rangle)\{\ldots\}\{\ldots\}\ \{Q\}$ — Seq(37, 19)
38. $\vdash \mathtt{dup} : \theta_2$ — see Fig. 8
39. $\cdot \vdash \{Q\}\ \left(\begin{array}{l}\langle (Fwd, Rewrt), Rte(\mathtt{dst})\rangle \leftarrow \mathtt{config\ dup};\\ \mathtt{if}(\langle Fwd(\mathtt{dst}^{\langle 1\rangle}), Rte(\mathtt{dst}^{\langle 2\rangle})\rangle)\{\ldots\}\{\ldots\};\\ \mathtt{if}(\langle Rewrt(\mathtt{dst}^{\langle 1\rangle}), Rte(\mathtt{dst})\rangle)\{\ldots\}\{\ldots\}\end{array}\right)\ \{Q\}$ — Bind(39, 37)

Manual proof in the logic

Relational **Program Logic** for **lens-configured programs**

